

ARM архитектура

Архитектурата ARM се отнася до архитектура на процесора, която еволюира от принципите на RISC дизайн и се използва във вградени системи.

Еволюция на ARM

ARM е семейство RISC-базирани микропроцесори и микроконтролери, проектирани от ARM Holdings, Кеймбридж, Англия. Компанията не произвежда процесори, а вместо това проектира микропроцесорни и многоядрени архитектури и ги лицензира на производителите. По-конкретно, ARM Holdings има два вида лицензирани продукти: процесори и процесорни архитектури. За процесори клиентът купува правата за използване на предоставения от ARM дизайн в собствените си чипове. За архитектура на процесора клиентът купува правата да проектира свой собствен процесор, съвместим с архитектурата на ARM. ARM чиповете са високоскоростни процесори, които са известни с малкия си размер на матрицата и ниските изисквания за мощност. Те се използват широко в смартфони и други преносими устройства, включително системи за игри, както и голямо разнообразие от потребителски продукти. ARM чиповете са процесорите в популярните iPod и iPhone устройства на Apple и се използват и в почти всички смартфони с Android. ARM е може би най-широко използваната архитектура на вградени процесори и наистина най-широко използваната процесорна архитектура от всякакъв вид в света. Произходът на ARM технологията може да се проследи до базираната в Англия компания Acorn Computers. В началото на 80-те години на миналия век Acorn получава договор от British Broadcasting Corporation (BBC) за разработване на нова микрокомпютърна архитектура за проекта за компютърна грамотност на BBC. Успехът на този договор позволи на Acorn да продължи с разработването на първия търговски RISC процесор, Acorn RISC Machine (ARM). Първата версия, ARM1, започва да функционира през 1985 г. и се използва за вътрешни изследвания и разработки, както и като копроцесор в машината на BBC. В този ранен етап Acorn използва компанията VLSI Technology, за да направи действителното производство на процесорните чипове. VLSI беше лицензиран да предлага на пазара чипа самостоятелно и имаше известен успех в накарването на други компании да използват ARM в своите продукти, особено като вграден процесор. Дизайнът на ARM отговаряше на нарастващата търговска нужда от високопроизводителен процесор с ниска консумация на енергия, малък размер и ниска цена за вградени приложения. Но по-нататъшното развитие беше извън обхвата на възможностите на Acorn. Съответно беше организирана нова компания с Acorn, VLSI и Apple Computer като основателни партньори, известна като ARM Ltd. Acorn RISC машината стана Advanced RISC Machines.¹²

Набор от инструкции

Наборът от инструкции ARM е много постоянен, проектиран за ефективно изпълнение на процесора и ефективно изпълнение. Всички инструкции са дълги 32 бита и следват постоянен формат. Това прави ARM ISA подходящ за внедряване в широка гама от продукти. Увеличаването на основния ARM ISA е наборът от инструкции Thumb, който е повторно кодирано подмножество на набора от инструкции ARM. Thumb е проектиран да повиши производителността на ARM реализации, които използват 16-битова или по-тясна шина за данни на паметта, и да позволи по-добра плътност на кода, отколкото се предоставя от набора от инструкции на ARM. Наборът от инструкции Thumb съдържа подмножество от 32-битовия набор от инструкции ARM, прекодиран в 16-битови инструкции. Текуща версия е Thumb-2.

ARM продукти

ARM Holdings лицензира редица специализирани микропроцесори и свързани технологии, но по-голямата част от тяхната продуктова линия е семейството на

микропроцесорните архитектури Cortex. Има три архитектури на Cortex, удобно обозначени с инициалите А, R и М.

Cortex-A и Cortex-A50

Cortex-A и Cortex-A50 са процесори за приложения, предназначени за мобилни устройства като смартфони и четци на електронни книги, както и потребителски устройства като цифрова телевизия и домашни шлюзове (например, DSL и кабелни интернет модеми). Тези процесори работят с по-висока тактова честота (над 1 GHz) и поддържат модул за управление на паметта (MMU), който е необходим за пълнофункционални ОС като Linux, Android, MS Windows и мобилни операционни системи. MMU е хардуерен модул, който поддържа виртуална памет и пейджинг чрез преобразуване на виртуални адреси във физически адреси; тази тема е разгледана в Глава 8. Двете архитектури използват както наборите инструкции ARM, така и Thumb-2; Основната разлика е, че Cortex-A е 32-битова машина, а Cortex-A50 е 64-битова машина.

Cortex-R

Cortex-R е проектиран да поддържа приложения в реално време, при които времето на събитията трябва да се контролира с бърза реакция на събития. Те могат да работят при доста висока тактова честота (например, 200MHz до 800MHz) и да имат много ниска латентност на реакцията. CortexR включва подобрения както на набора от инструкции, така и на организацията на процесора, за да поддържа дълбоко вградени устройства в реално време. Повечето от тези процесори нямат MMU; ограничените изисквания за данни и ограничения брой едновременни процеси елиминират необходимостта от сложна хардуерна и софтуерна поддръжка за виртуална памет. Cortex-R има модул за защита на паметта (MPU), кеш и други функции на паметта, предназначени за индустриални приложения. MPU е хардуерен модул, който забранява на една програма в паметта случайно да осъществява достъп до памет, назначена на друга активна програма. С помощта на различни методи се създава защитна граница около програмата и инструкциите в програмата са забранени да препращат към данни извън тази граница. Примери за вградени системи, които биха използвали Cortex-R, са автомобилни спирачни системи, контролери за масово съхранение и устройства за работа в мрежа и печат.

CORTEX-M

Процесорите от серията Cortex-M са разработени предимно за домейна на микроконтролерите, където нуждата от бързодействие, целенасочено управление на прекъсванията се съчетава с нужда от малка логика и ниска консумация на енергия. Както при Cortex-R, серията Cortex-M има MPU, но няма MMU. Cortex-M използва само набора от инструкции Thumb-2. Пазарът на Cortex-M включва IoT устройства, безжични сензорни/задвижващи мрежи, използвани във фабрики и други предприятия, автомобилна електроника и т.н. В момента има четири версии на серията Cortex:

Cortex-M0:

Проектиран за 8- и 16-битови приложения, този модел е прост, има ниска цена и консумира много ниска мощност. Оптимизиран е за малък размер проста силиконова матрица за употреба в чипове с ниска цена. Cortex-M0+ е подобрена и енергийно по-ефективна версия на M0.

Cortex-M3:

Проектиран за 16- и 32-битови приложения, този модел набляга на производителността и енергийната ефективност. Той разполага с функции за отстраняване на грешки и проследяване действието на програмата, които позволяват на софтуерните разработчици да работят по-ефективно.

Cortex-M4:

Този модел предоставя всички функции на Cortex-M3, с допълнителни инструкции за поддръжка на задачи за цифрова обработка на сигнала. В този текст ще използваме предимно ARM Cortex-M3 като наш примерен вграден системен процесор. Той е най-подходящият от всички модели ARM за използване на микроконтролери с общо предназначение. Cortex-M3 се използва от различни производители на продукти за микроконтролери. Водещи производители комбинират процесора Cortex-3 с флеш, SRAM и множество периферни устройства, за да осигурят конкурентно предложение на цена от \$1. Фигура 1.16 представя блокова схема на микроконтролера EFM32 от Silicon Labs. Тя показва подробности за процесора Cortex-M3 и основните компоненти.

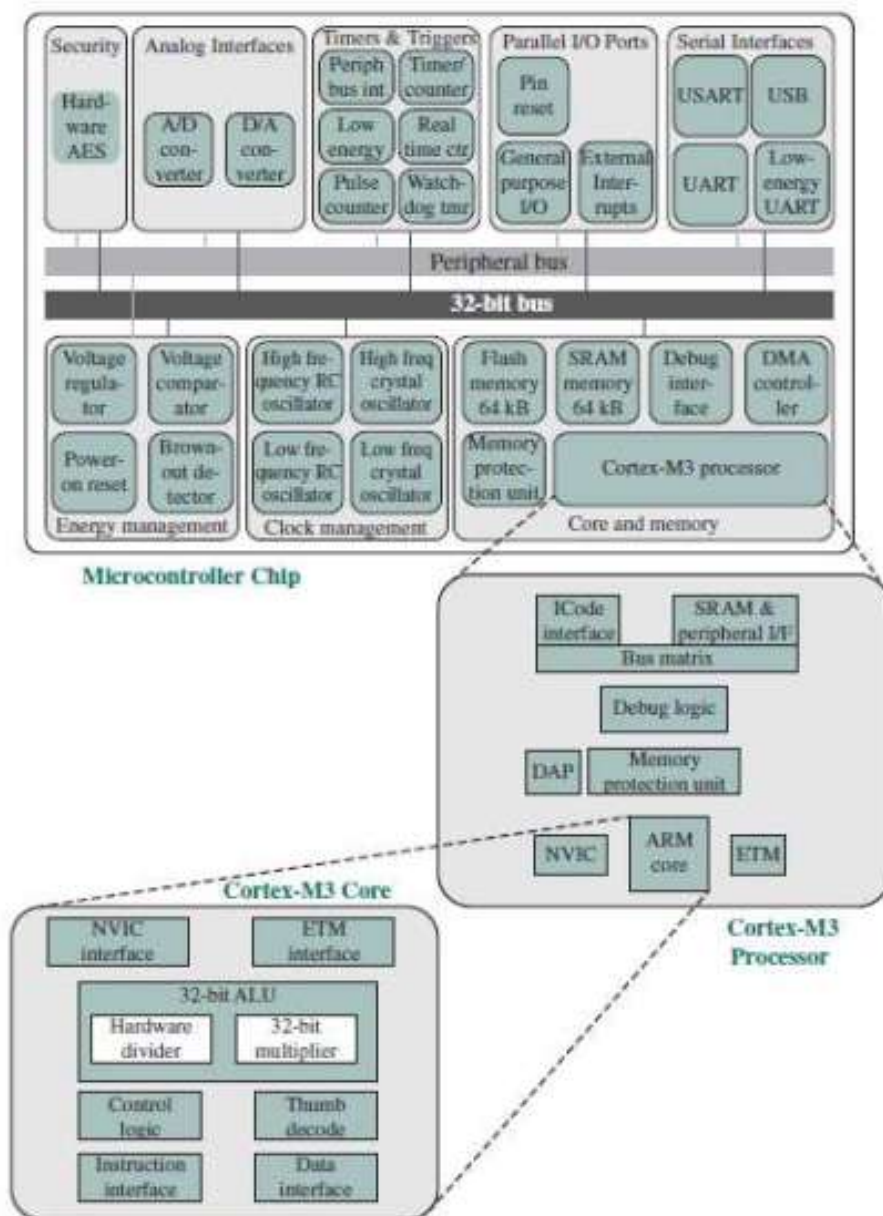


Figure 1.16 Typical Microcontroller Chip Based on Cortex-M3.

Ядрото Cortex-M3 използва отделни шини за инструкции и данни. Тази архитектура се нарича Харвардска, за разлика от архитектурата на фон Нойман, която използва същите сигнални шини и памет за инструкции и за данни. Така Cortex-M3 е в състояние да чете едновременно инструкция и данни от паметта и изпълнява много

операции паралелно, ускорявайки изпълнението на приложението. Ядрото съдържа декодер за Thumb инструкции, усъвършенстван ALU с поддръжка за хардуерно умножение и деление, логика за управление и интерфейси към другите компоненти на процесора. Има интерфейс към контролера за вложени векторни прекъсвания (NVIC) и модула за вградени макроклетки за проследяване (ETM).

Ядрото е част от модула, наречен Cortex-M3 процесор. Процесорът включва още следните елементи:

NVIC: Предоставя конфигурируеми възможности за обработка на прекъсвания на процесора. Той улеснява обработката на изключения и прекъсвания с ниска латентност и контролира управлението на захранването.

ETM: Допълнителен компонент за отстраняване на грешки, който позволява реконструкция на изпълнението на програмата. ETM е бърз инструмент за отстраняване на грешки с ниска мощност, който поддържа само проследяване на инструкции.

Порт за достъп за отстраняване на грешки (DAP): Той осигурява интерфейс за външен достъп за отстраняване на грешки до процесора.

Логика за отстраняване на грешки: Основната функционалност за отстраняване на грешки включва спиране на процесора, едноетапен достъп до регистъра на процесора, неограничен брой софтуерни точки на прекъсване и пълен достъп до системната памет.

■ **ICode интерфейс:** Извлича инструкции от пространството на кодовата памет.

■ **SRAM и периферен интерфейс:** Интерфейс за четене/запис към паметта за данни и периферните устройства.

■ **Матрична шина:** Свързва интерфейсите за ядро и отстраняване на грешки към външни шини на микроконтролера.

■ **Устройство за защита на паметта:** Защищава критичните данни, използвани от операционната система, от потребителски приложения, разделяйки задачите за обработка, като забранява достъпа до данните на другия, забранява достъпа до регионите на паметта, позволява на регионите на паметта да бъдат дефинирани като само за четене и открива неочакван достъп до паметта което потенциално може да счупи системата. Горната част на фигура 1.16 показва блоковата схема на типичен микроконтролер, изграден с Cortex-M3, в този случай микроконтролер EFM32. Този микроконтролер се предлага на пазара за използване в голямо разнообразие от устройства, включително измерване на енергия, газ и вода; алармени и охранителни системи; устройства за промишлена автоматизация; устройства за домашна автоматизация; интелигентни аксесоари; и устройства за здраве и фитнес.

Силиконовият чип се състои от 10 основни области:

■ **Ядро и памет:** Този регион включва процесор Cortex-M3, статична RAM (SRAM) памет за данни и флаш памет за съхранение на програмни инструкции и непроменливи данни от приложението. Флаш паметта е енергонезависима (данните не се губят при изключване на захранването) и затова е идеална за тази цел. SRAM съхранява променливи данни. Тази област включва и интерфейс за отстраняване на грешки, който улеснява препрограмирането и актуализирането на системата на място.

■ **Паралелни I/O портове:** Конфигурируеми за различни паралелни I/O схеми.

■ **Серийни интерфейси:** Поддържа различни серийни I/O схеми.

■ **Аналогови интерфейси:** Аналогово-цифрова и цифрово-аналогова логика за поддръжка на сензори и задвижващи механизми.

■ **Таймери и тригери:** Следи времето и брой събития, генерира изходни импулсни поредици за действия тригери в определен времеви интервал в други периферни устройства.

- **Управление на часовника:** Контролира часовниците и осцилаторите на чипа. Използват се множество часовници и осцилатори за минимизиране на консумацията на енергия и осигуряване на кратко време за стартиране.
- **Управление на енергията:** Управлява различните нискоенергийни режими на работа на процесора и периферните устройства, за да осигури управление в реално време на енергийните нужди, така че да се сведе до минимум консумацията на енергия.
- **Сигурност:** Чипът включва хардуерна реализация на Advanced Encryption Standard (AES).
- **32-битова шина:** Свързва всички компоненти на чипа.
- **Периферна шина:** Мрежа, която позволява на различните периферни модули да комуникират директно един с друг, без да включва процесора. Това поддържа критична за времето операция и намалява излишните разходи за софтуер. Има много прилики и същата обща йерархична структура като РС. Обаче най-горното ниво на компютърната система с микроконтролер е един чип, докато за многоядрен компютър най-горното ниво е дънна платка, съдържаща определен брой чипове. Друга разлика е, че няма кеш, нито в процесора Cortex-M3, нито в микроконтролера като цяло, което играе важна роля, ако кодът или данните се намират във външна памет. Кешът значително подобрява производителността, когато се използва външна памет. Такива режимни разходи не са необходими за микроконтролер.

Управление на ARM паметта

ARM предоставя гъвкава системна архитектура за виртуална памет, която може да бъде съобразена с нуждите на дизайнера на вградената система.

Организация на системата на паметта

Фигура 8.22 предоставя общ преглед на хардуера за управление на паметта в ARM за виртуална памет. Хардуерът за превод на виртуална памет използва едно или две нива на таблици за превод от виртуални към физически адреси, както е обяснено по-нататък. Буферът (TLB) е кеш на последните записи в таблицата на страниците. Ако записът е наличен в TLB, тогава TLB директно изпраща физически адрес към основната памет за операция за четене или запис.

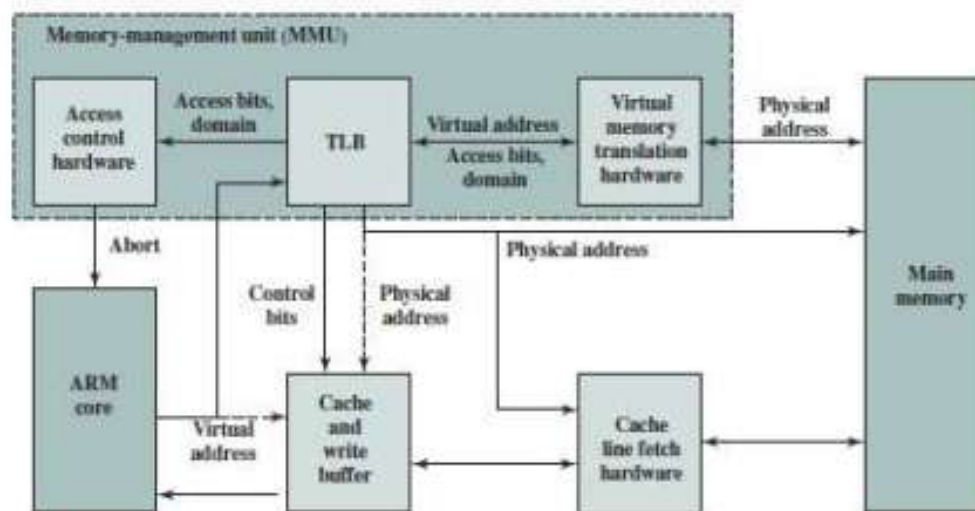


Figure 8.22 ARM Memory System Overview

Данните се обменят между процесора и основната памет чрез кеша..

Превод на адреси на виртуална памет

ARM поддържа достъп до памет въз основа на секции или страници:

- **Суперсекции** (по избор): Състоят се от 16-MB блокове основна памет.
- **Секции**: Състоят се от 1-MB блокове основна памет.
- **Големи страници**: Състоят се от 64-kB блокове основна памет.
- **Малки страници**: Състоят се от 4-kB блокове основна памет. Поддържат се раздели и суперсекции, за да се позволи картографиране на голям регион на паметта, като се използва само един запис в TLB. Допълнителните механизми за контрол на достъпа са разширени в малки страници до 1kB подстраници и в големи страници до 16kB подстраници. Таблицата за превод, съхранявана в основната памет, има две нива:
- **Таблица от ниво 1**: Съдържа дескриптори от ниво 1, които съдържат основния адрес и свойствата на трансляция за раздел и суперсекция; и свойства за превод и указатели към таблица от ниво 2 за голяма или малка страница.
- **Таблица от ниво 2**: Съдържа дескриптори от ниво 2, които съдържат основния адрес и свойствата на превода за малка страница или голяма страница. Таблица от ниво 2 изисква 1 kB памет. Устройството за управление на паметта (MMU) превежда виртуалните адреси, генерирани от процесора, във физически адреси за достъп до основната памет, а също така извлича и проверява разрешението за достъп. Преводите възникват в резултат на пропуск на TLB и започват с извличане от първо ниво. Достъпът, картографиран на раздели, изисква само извличане от първо ниво, докато достъпът с картографирана страница също изисква извличане от второ ниво.

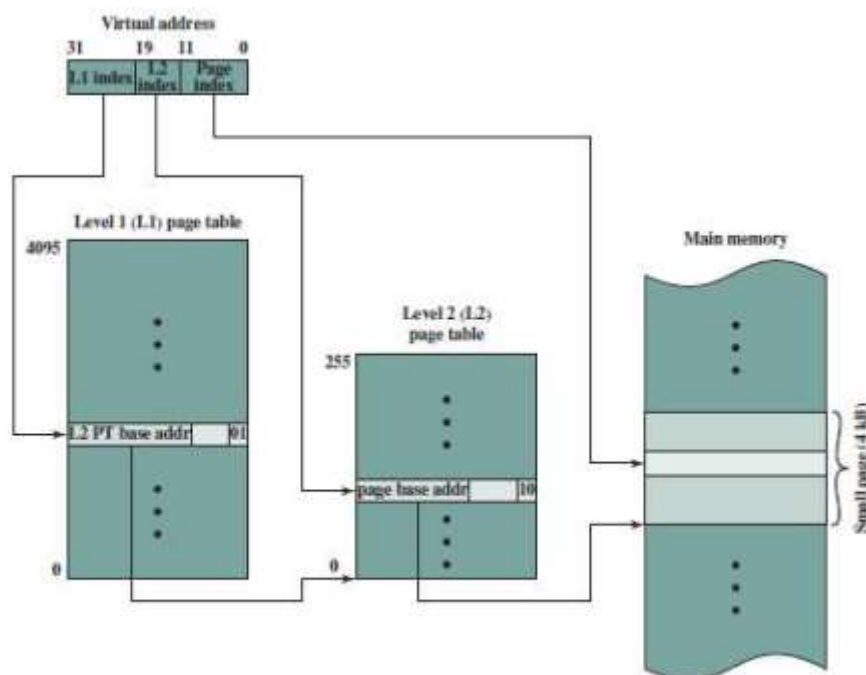


Figure 8.23 ARM Virtual Memory Address Translation for Small Pages

Фигура 8.23 показва процеса на преобразуване на адреси на две нива за малки страници. Има една таблица на страници от ниво 1 (L1) с 4K 32-битови записи. Всеки запис от L1 сочи към таблица на страници от ниво 2 (L2) с 256 32-битови записа. Всеки от входовете L2 сочи към страница от 4 kB в основната памет. 32-битовият виртуален адрес се интерпретира по следния начин: Най-значимите 12 бита са индекс в таблицата на страниците L1. Следващите 8 бита са индекс в съответната таблица на страници L2. Най-малко значимите 12 бита индексират байт в съответната страница в основната памет. Подобна процедура за търсене на две страници се използва за големи страници. За раздели и суперсекции се изисква само търсене в таблицата на страниците L1.

Регион на паметта може да бъде определен като без достъп, само за четене или за четене-запис. Освен това регионът може да бъде определен само като привилегирован достъп, запазен за използване от ОС, а не от приложения. ARM също така използва концепцията за домейн, който е колекция от секции и/или страници, които имат определени разрешения за достъп. Архитектурата на ARM поддържа 16 домейна. Функцията на домейна позволява на множество процеси да използват едни и същи таблици за превод, като същевременно поддържат известна защита един от друг. Поддържат се два вида достъп до домейн:

■ **Клиенти:** Потребители на домейни (изпълнение на програми и данни за достъп), които трябва да спазват разрешенията за достъп на отделните секции и/или страници, които съставляват този домейн.

■ **Мениджъри:** Контролира поведението на домейна (текущите раздели и страници в домейна и достъпа до домейна) и заобикаля разрешенията за достъп за записи в таблицата в този домейн. Една програма може да бъде клиент на някои домейни и мениджър на някои други домейни и да няма достъп до останалите домейни. Това позволява много гъвкава защита на паметта за програми, които имат достъп до различни ресурси на паметта.

Типове данни ARM

ARM процесорите поддържат типове данни с дължина 8 (байт), 16 (половин дума) и 32 (дума) бита. Обикновено достъпът до половин дума трябва да бъде подравнен на половината дума, а достъпът до думи трябва да бъде подравнен по думите. За опити за неподравнен достъп архитектурата поддържа три алтернативи.

■ **Случай по подразбиране:** – Адресът се третира като съкратен, като адресните битове [1:0] се третират като нула за достъп до думи, а адресният бит [0] се третира като нула за достъп до половин дума.

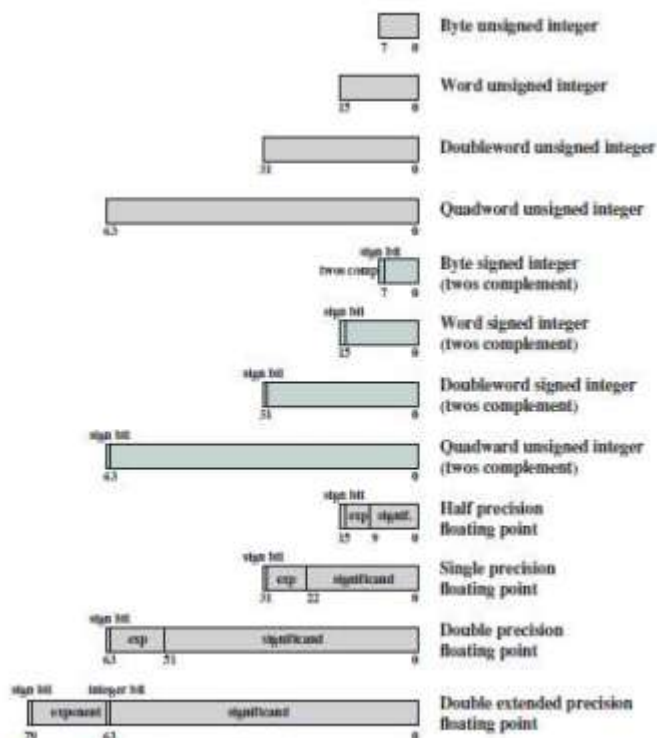


Figure 12.4 x86 Numeric Data Formats

Инструкциите за ARM за зареждане на една дума са архитектурно дефинирани да завъртат надясно подравнените по думи данни, прехвърлени от неподравнен по думи

адрес с един, два или три байта в зависимост от стойността на двата най-малко значими адресни бита.

■ **Проверка на изравняването:** Когато е зададен съответния контролен бит, сигнал за прекратяване на данните показва грешка в изравняването за опит за неизравнен достъп.

■ **Неизравнен достъп:** Когато тази опция е активирана, процесорът използва един или повече достъпи до паметта, за да генерира необходимото прехвърляне на съседни байтове прозрачно към програмиста.

И за трите типа данни (байт, полудума и дума) се поддържа интерпретация без знак, в която стойността представлява беззнаково, неотрицателно цяло число. И трите типа данни могат да се използват и за целите числа със знак, в допълнителен код. По-голямата част от реализациите на ARM процесорите не предоставят хардуер с плаваща запетая, което спестява енергия и площ. Ако в такива процесори се изисква аритметика с плаваща запетая, тя трябва да бъде реализирана софтуерно. ARM поддържа допълнителен копроцесор с плаваща запетая, който поддържа типовете данни с единична и двойна точност, дефинирани в IEEE 754.

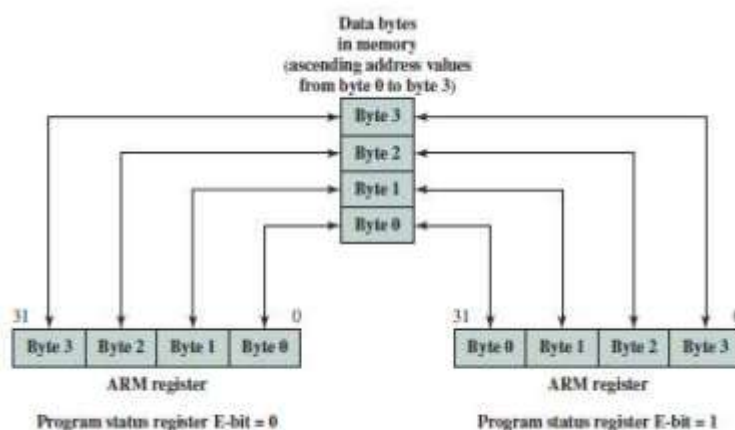


Figure 12.5 ARM Endian Support – Word Load/Store with E-Bit

ПОДДРЪЖКА НА ENDIAN

Битът за състояние (Е-бит) в регистъра за управление на системата се задава и изчиства под програмен контрол с помощта на инструкцията SETEND. Е-битът определя кой байт за зареждане и съхранение на данни. Фигура 12.5 илюстрира функционалността, свързана с Е-бита за операция за зареждане или съхранение на думи. Този механизъм дава възможност за ефективно динамично зареждане/съхранение на данни за системни дизайнери, които знаят, че трябва да имат достъп до структури от данни в обратен порядък на порядъка на порядъка на тяхната ОС/среда. Адресът на всеки байт данни е фиксиран в паметта. Въпреки това, байтовата лента в регистъра е различна.

Типове операции на ARM

Архитектурата на ARM предоставя голям брой операции. Основните категории инструкции са:

■ Инструкции за зареждане и съхранение:

В ARM архитектурата само инструкциите за зареждане и съхранение имат достъп до местата на паметта; аритметичните и логическите инструкции се изпълняват само върху регистри и непосредствени стойности, кодирани в инструкцията. Това ограничение е характерно за RISC дизайна. Архитектурата на ARM поддържа два широки типа инструкции, които зареждат или съхраняват стойността на един регистър или двойка регистри от или към паметта:

- (1) зареждане или съхраняване на 32-битова дума или байт без знак ;
- (2) зареждане или съхраняване на 16-битова полудума без знак и зареждане на 16-битова полудума или 8-битов байт със знак.

■ **Инструкции за разклонение:**

ARM поддържа инструкция за разклонение, което позволява условен преход в програмата напред или назад до 32 MB адреса. Извикване на подпрограма може да се извърши от вариант на стандартната инструкция за условен преход. Освен че позволява разклоняване напред или назад до 32 MB, инструкцията за клон с връзка (BL) запазва адреса на инструкцията след разклонението (адреса за връщане) в LR (R14). Разклонението се определят от 4-битово поле за условие в инструкцията.

■ **Инструкции за обработка на данни:**

Тази категория включва логически инструкции (И, ИЛИ, изкл. или), инструкции за добавяне и изваждане и инструкции за тестване и сравнение.

■ **Инструкции за умножение:**

Инструкциите за целочислено умножение работят с операнди на дума или половин дума и могат да дадат нормални или дълги резултати. Например, има инструкция за умножение, която приема два 32-битови операнда и дава 64-битов резултат.

■ **Инструкции за паралелно събиране и изваждане:**

В допълнение към нормалните инструкции за обработка на данни и умножение, има набор от паралелни инструкции за събиране и изваждане, в които части от два операнда се оперират паралелно. Например, ADD16 добавя горните полудуми на два регистъра, за да образува горната половин дума на резултата, и добавя долните полудуми на същите два регистъра, за да образува долната половин дума на резултата. Тези инструкции са полезни в приложения за обработка на изображения, подобно на инструкциите x86 MMX.

■ **Инструкции за разширяване:** Има няколко инструкции за разопаковане на данни. Те разширяват байтовете до полудуми или думи и полудумите до думи.

■ **Инструкции за достъп до регистъра на състоянието:**

ARM предоставя възможност за четене и за запис на части от регистъра на състоянието. с кодове на условия. Има 4 флага за условия, които се съхраняват в регистъра на състоянието на програмата: N, Z, C и V (отрицателен, нула, пренос и препълване), със значения по като S, Z, C и V флагове в архитектурата Intel x86. Тези четири флага представляват код на условие в ARM.

Таблица 12.11 показва комбинацията от условия, за които е дефинирано условно изпълнение. Има две особености при използването на кодове за условие в ARM:

1. Всички инструкции, а не само инструкции за разклонение, включват поле за код на условие, което означава, че почти всички инструкции могат да бъдат изпълнени условно. Всяка комбинация от настройки на флага с изключение на 1110 или 1111 в полето с код на условието на инструкцията означава, че инструкцията ще бъде изпълнена само ако условието е изпълнено.
2. Всички инструкции за обработка на данни (аритметични, логически) включват S бит, който означава дали инструкцията актуализира флаговете на условието.

Table 12.11 ARM Conditions for Conditional Instruction Execution

Code	Symbol	Condition Tested	Comment
0000	EQ	$Z = 1$	Equal
0001	NE	$Z = 0$	Not equal
0010	CS/HS	$C = 1$	Carry set/unsigned higher or same
0011	CC/LO	$C = 0$	Carry clear/unsigned lower
0100	MI	$N = 1$	Minus/negative
0101	PL	$N = 0$	Plus/positive or zero
0110	VS	$V = 1$	Overflow
0111	VC	$V = 0$	No overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Unsigned higher
1001	LS	$C = 0 \text{ OR } Z = 1$	Unsigned lower or same
1010	GE	$N = V$ $[(N = 1 \text{ AND } V = 1) \text{ OR } (N = 0 \text{ AND } V = 0)]$	Signed greater than or equal
1011	LT	$N \neq V$ $[(N = 1 \text{ AND } V = 0) \text{ OR } (N = 0 \text{ AND } V = 1)]$	Signed less than
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Signed greater than
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Signed less than or equal
1110	AL	–	Always (unconditional)
1111	–	–	This instruction can only be executed unconditionally

Използването на условно изпълнение и настройка на флаговете за условия помага за проектирането на по-кратки програми, които използват по-малко памет. Всички инструкции включват 4 бита за код на условието. По-малко битове в 32-битовата инструкция се ползват за код на операцията и операнди. Тъй като ARM е RISC дизайн и основно адресира регистри, това е разумен компромис

Режими на ARM адресиране

Обикновено RISC машината, за разлика от CISC машината, използва прост и сравнително ясен набор от режими на адресиране. Архитектурата ARM се отклонява донякъде от тази традиция, като предоставя относително богат набор от режими на адресиране. Тези режими са най-удобно класифицирани по отношение на вида на инструкцията.

ИНСТРУКЦИИ ЗА ЗАРЕЖДАНЕ/СЪХРАНЯВАНЕ

Инструкциите за зареждане и съхранение са единствените инструкции, които се обръщат към паметта. Това винаги се прави индиректно чрез основен регистър плюс изместване. Има три алтернативи по отношение на индексването (Фигура 13.3):

■ Отместване:

За този метод на адресиране индексването не се използва. Стойност на отместване се добавя или изважда от стойността в основния регистър, за да се образува адресът на паметта. Като пример Фигура 13.3а илюстрира този метод с инструкцията на асемблер STRB r0, [r1, #12]. Това е инструкцията за съхраняване на байтове.

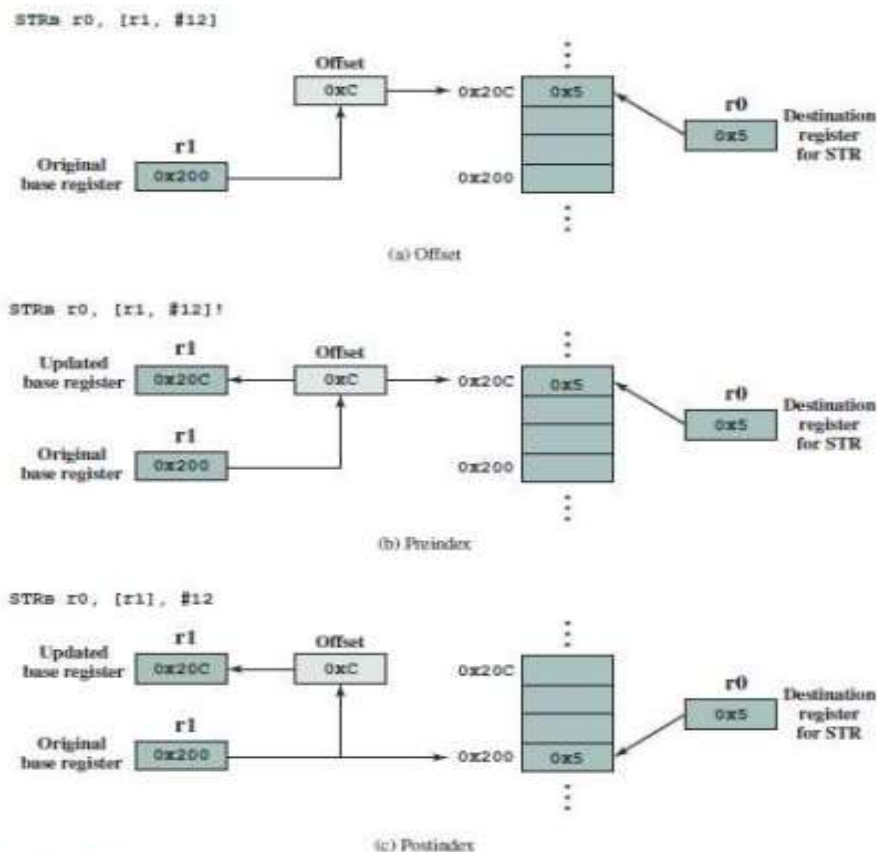


Figure 13.3 ARM Indexing Methods

В този случай основният адрес е в регистър `r1` и изместването е непосредствена стойност от десетичен знак 12. Полученият адрес (база плюс отместване) е мястото, където трябва да се съхранява най-малко значимият байт от `r0`.

■ **Прединдексиране:** Адресът на паметта се формира по същия начин, както при адресиране с отместване. Адресът на паметта също се записва обратно в основния регистър. С други думи, стойността на основния регистър се увеличава или намалява със стойността на отместване. Фигура 13.3b илюстрира този метод с инструкцията на асемблер `STRB r0, [r1, #12]!`. Удивителният знак означава предварително индексирание.

■ **Слединдексиране:** Адресът на паметта е стойността на основния регистър. Отместване се добавя или изважда от стойността на основния регистър и резултатът се записва обратно в основния регистър. Фигура 13.3c илюстрира този метод с инструкцията на асемблер `STRB r0, [r1], #12`. Това, което ARM нарича основен регистър, действа като индексен регистър за адресиране преди и след индексирание. Стойността на отместването може да бъде или непосредствена стойност, съхранена в инструкцията, или може да се намира в друг регистър. Ако стойността на отместването е разположена в регистър, има друга полезна функция: адресиране на мащабиран регистър. Стойността в регистъра за отместване се мащабира от един от операторите на смяна: логическо отместване наляво, логическо изместване надясно, аритметично изместване надясно, завъртане надясно или разширено завъртане надясно (което включва бит за пренасяне в ротацията). Размерът на смяната се посочва като непосредствена стойност в инструкцията.

ИНСТРУКЦИИ ЗА ОБРАБОТКА НА ДАННИ

Инструкциите за обработка на данни използват или регистрово адресиране, или смес от регистрово и непосредствено адресиране. При регистрово адресиране, стойността в

един от операндите на регистъра може да бъде мащабирана с помощта на един от петте оператора на смяна, определени в предходния параграф.

ИНСТРУКЦИИ ЗАРАЗКЛОНЕНИЕ

Единствената форма на адресиране за инструкции на разклонение е непосредственото адресиране. Инструкцията за разклонение съдържа 24-битова стойност. За изчисляване на адреса, тази стойност се измества наляво с 2 бита, така че адресът да е изравнен на границата на дума. По този начин ефективният адресен диапазон на прехода е ± 32 MB от програмния брояч.

ЗАРЕЖДАНЕ/СЪХРАНЯВАНЕ С МНОЖЕСТВЕНО АДРЕСИРАНЕ

Зареждане Множество инструкции зареждат подмножество (евентуално всички) от регистрите с общо предназначение от паметта. **Съхраняване** Множество инструкции съхраняват подмножество (евентуално всички) от регистри с общо предназначение в паметта. Списъкът с регистри за зареждане или съхранение е посочен в 16-битово поле в инструкцията, като всеки бит съответства на един от 16-те регистъра.

Зареждане и съхранение

Множество режими на адресиране създават последователен диапазон от адреси на паметта. Регистърът с най-нисък номер се съхранява на най-ниския адрес на паметта, а регистърът с най-голям номер се съхранява на най-високия адрес на паметта. Използват се четири режима на адресиране (Фигура 13.4): пред- и след- инкрементация, пред- и след- декрементация.

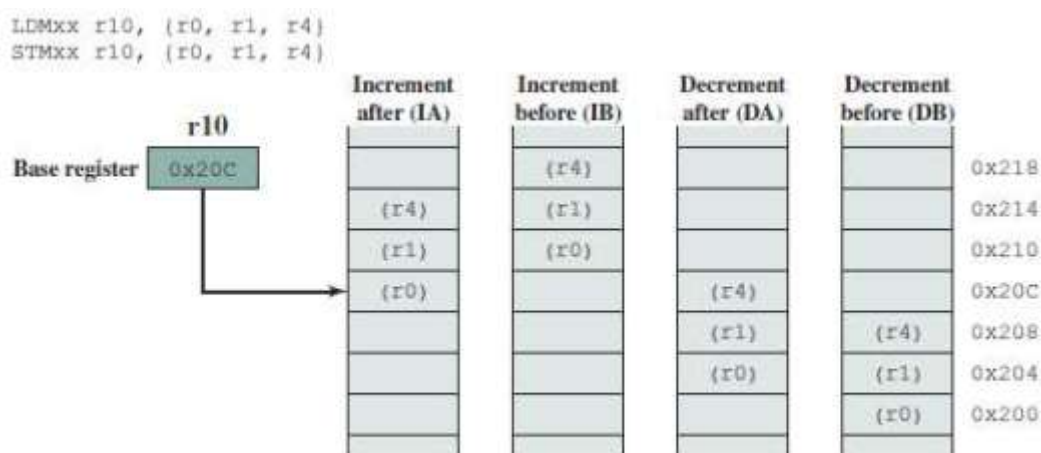


Figure 13.4 ARM Load/Store Multiple Addressing

Базовият регистър определя адрес на основната памет, където стойностите на регистъра се съхраняват или зареждат във възходяща (нарастване) или низходяща (намаляване) местоположения на думите. Увеличаването или намаляването започва преди или след първото обръщение към паметта. Тези инструкции са полезни за блоково зареждане или съхранение, операции със стека и последователности за излизане от процедури.

Формат на инструкциите за X86 и ARM

Интел x86 има различни формати на инструкциите. Общо е само полето код на операцията. Фигура 13.9 илюстрира общия формат на инструкциите. Инструкциите се състоят от 0 до 4 незадължителни префикса на инструкцията, 1- или 2-байтов операционен код, незадължителен спецификатор на адрес (който се състои от байта ModR/M и базовия байт на индекса на мащаба), незадължително изместване и незадължително поле за непосредствен операнд.

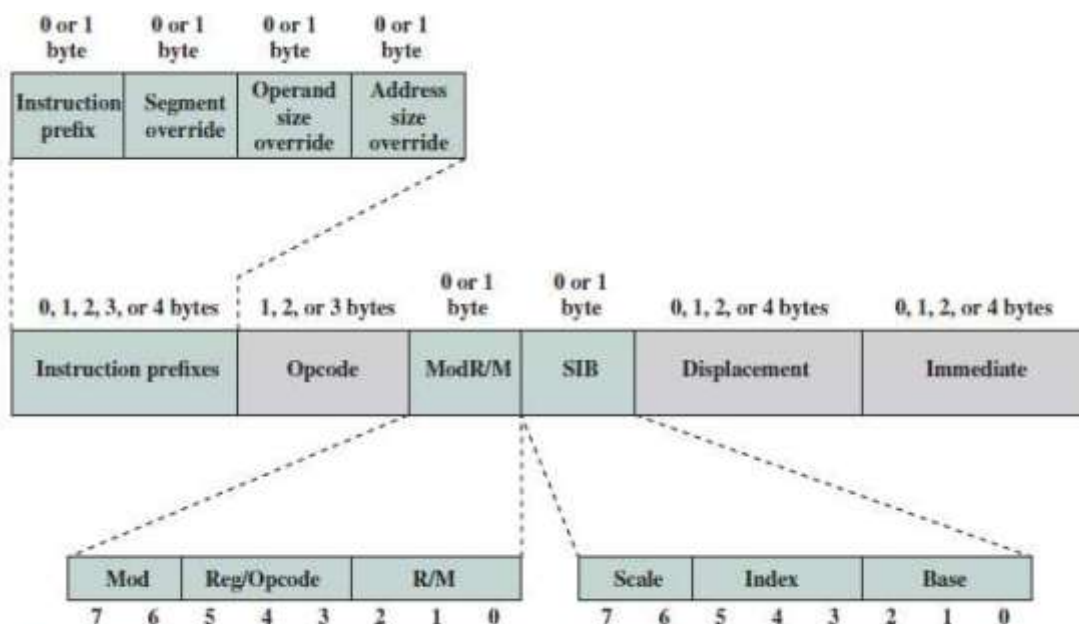


Figure 13.9 x86 Instruction Format

Формати на инструкциите на ARM

Всички инструкции в архитектурата на ARM са дълги 32 бита и следват постоянен формат (Фигура 13.10). Първите четири бита на инструкцията са кодът на условието. Както беше обсъдено в Глава 12, практически всички ARM инструкции могат да бъдат изпълнени условно. Следващите три бита определят общия тип инструкция. За повечето инструкции, различни от инструкциите за разклоняване, следващите пет бита представляват код на операцията и/или битове модификатори за операцията. Останалите 20 бита са за адресиране на операнда. Редовната структура на форматите на инструкциите улеснява работата на единиците за декодиране на инструкции.

Непосредствени константи.

За да се постигне по-голям обхват от непосредствени стойности, непосредственият формат за обработка на данни определя както непосредствена стойност, така и стойност за завъртане. 8-битовата непосредствена стойност се разширява до 32 бита и след това се завърта надясно с брой битове, равен на удвоената 4-битова стойност за завъртане. Няколко примера са показани на Фигура 13.11.

НАБОР THUMB ИНСТРУКЦИИ

Наборът от инструкции Thumb е повторно кодирано подмножество от набора от инструкции ARM. Thumb е проектиран да повиши производителността на ARM реализации, които използват 16-битова или по-тъсна шина за данни на паметта и да позволи по-добра плътност на кода от предоставената от ARM инструкцията както за 16-битови, така и за 32-битови процесори. Наборът от инструкции Thumb е създаден чрез анализиране на 32-битовия набор от инструкции ARM и извличане на най-подходящия 16-битов набор от инструкции, като по този начин се намалява размерът на кода. Спестяванията се постигат по следния начин: 1. Thumb инструкциите са безусловни, така че полето за код на условието не се използва. Също така, всички аритметични и логически инструкции Thumb актуализират флаговете на условието, така че битът за флаг за актуализиране не е необходим. Спестяват се 5 бита.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Data processing immediate shift	cond	0	0	0	opcode				S	Rn				Rd				shift amount				shift	0	Rm								
Data processing register shift	cond	0	0	0	opcode				S	Rn				Rd				Rs				0	shift	1	Rm							
Data processing immediate	cond	0	0	1	opcode				S	Rn				Rd				rotate				immediate										
Load/store immediate offset	cond	0	1	0	P	U	B	W	L	Rn				Rd				immediate														
Load/store register offset	cond	0	1	1	P	U	B	W	L	Rn				Rd				shift amount				shift	0	Rm								
Load/store multiple	cond	1	0	0	P	U	S	W	L	Rn				register list																		
Branch/branch with link	cond	1	0	1	L	24-bit offset																										

S = For data processing instructions, signifies that the instruction updates the condition codes

S = For load/store multiple instructions, signifies whether instruction execution is restricted to supervisor mode

P, U, W = bits that distinguish among different types of addressing mode

B = Distinguishes between an unsigned byte (B=1) and a word (B=0) access

L = For load/store instructions, distinguishes between a Load (L=1) and a Store (L=0)

L = For branch instructions, determines whether a return address is stored in the link register

Figure 13.10 ARM Instruction Formats

2. Thumb има само подмножество от операциите в пълния набор от инструкции и използва само 2-битово поле за код на операцията плюс поле от 3-битов тип. Спестява 2 бита.

3. Останалите спестявания от 9 бита идват от намаляване на спецификациите на операнда. Например инструкциите Thumb препращат само към регистри от r0 до r7, така че за препратки към регистъра са необходими само 3 бита, а не 4 бита. Непосредствените стойности не включват 4-битово поле за завъртане.

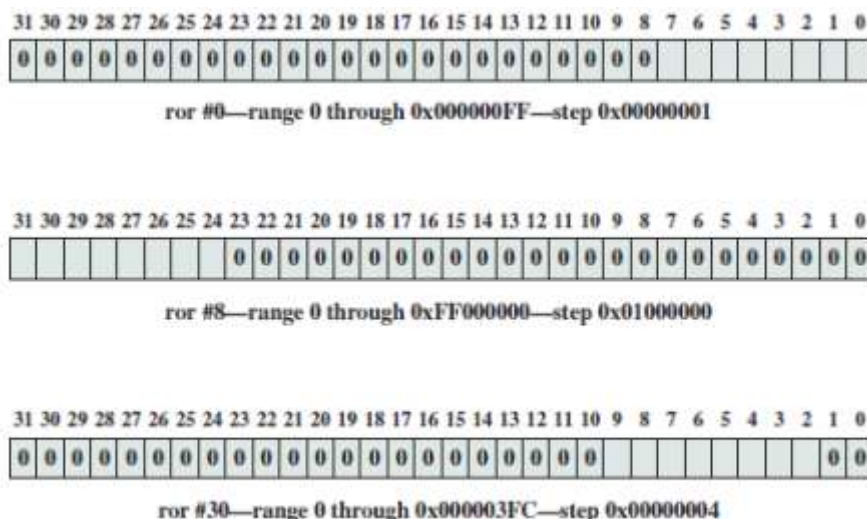


Figure 13.11 Examples of Use of ARM Immediate Constants

ARM процесорът може да изпълнява програма, състояща се от смесица от Thumb инструкции и 32-битови ARM инструкции. Един бит в регистъра за управление на процесора определя кой тип инструкция се изпълнява в момента. Фигура 13.12 показва пример. Фигурата показва както общия формат, така и специфичен екземпляр на инструкция както в 16-битов, така и в 32-битов формат.

THUMB-2 НАБОР ИНСТРУКЦИИ

С въвеждането на набора от инструкции Thumb, от потребителя се изискваше да комбинира набори от инструкции чрез компилиране на критичен за производителността код към ARM, а останалата част към Thumb. Това ръчно смесване на код изисква допълнителни усилия и е трудно да се постигнат оптимални резултати. За да преодолее тези проблеми, ARM разработи набора от инструкции Thumb-2, който е единственият набор от инструкции, наличен в продуктите на Cortex-Mmicrocontroller. Thumb-2 е основно подобрение на архитектурата на набора от инструкции Thumb (ISA). Той въвежда 32-битови инструкции, които могат да се смесват свободно с по-старите 16-битови инструкции Thumb. Тези нови 32-битови инструкции покриват почти цялата функционалност на набора от инструкции ARM. Най-важната разлика между Thumb ISA и ARM ISA е, че повечето 32-битови Thumb инструкции са безусловни, докато почти всички ARM инструкции могат да бъдат условни. Въпреки това Thumb-2 въвежда нова If-Then(IT) инструкция, която предлага голяма част от функциите на полето за условие в ARM инструкциите. Thumb-2 осигурява обща плътност на кода, сравнима с Thumb, а нива на производителност, близки до ARM ISA. Преди Thumb-2 разработчиците избираха между Thumb за малък размер и ARM за висока производителност. Сравнението на набора от инструкции Thumb-2 с ARM и оригиналните набори от инструкции Thumb за бързодействието след компилиране и изпълнение за вградени микропроцесори (EEMBC) при трите набора инструкции дава следните резултати:

- С компилатори, оптимизирани за производителност, размерът на Thumb-2 е с 26% по-малък от ARM и малко по-голям от оригиналния Thumb.
- С компилатори, оптимизирани за дължина на кода, размерът на Thumb-2 е с 32% по-малък от ARM и малко по-малък от оригиналния Thumb.

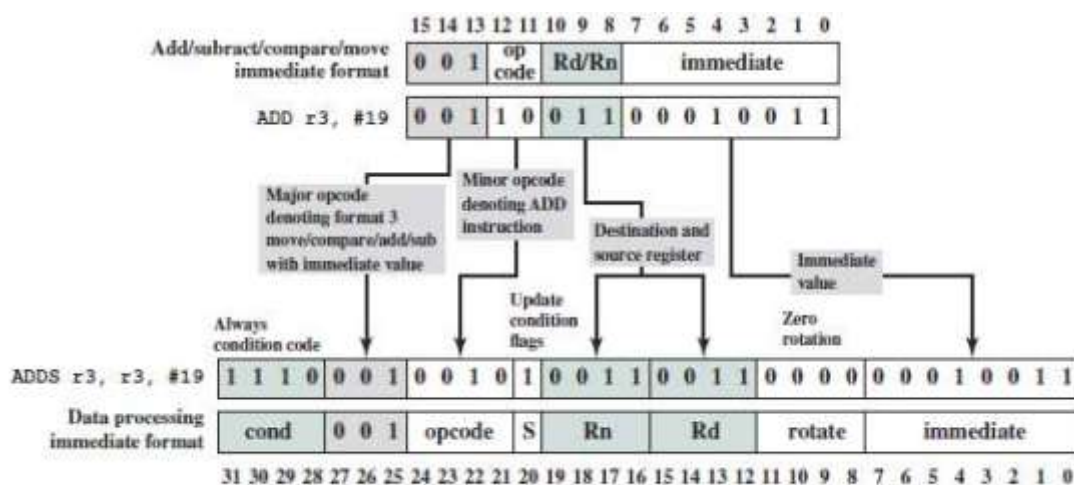


Figure 13.12 Expanding a Thumb ADD Instruction into its ARM Equivalent

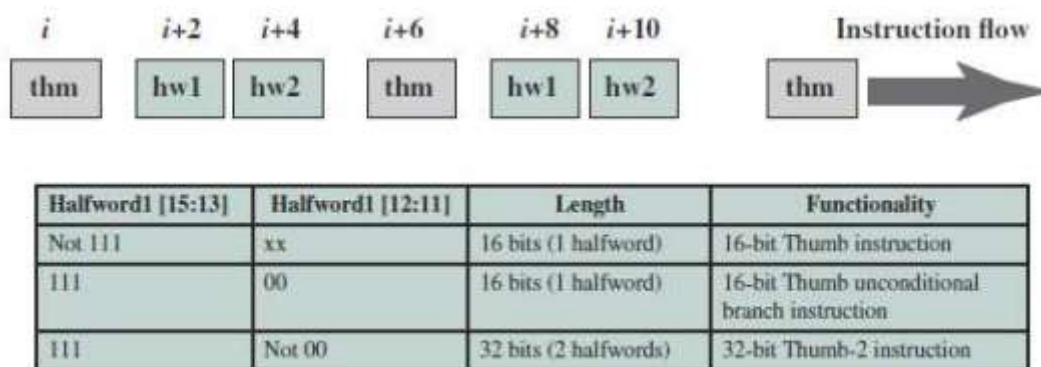


Figure 13.13 Thumb-2 Encoding

■ С компилатори, оптимизирани за производителност, производителността на Thumb-2 в пакета за сравнителни тестове е 98% от производителността на ARM и 125% от оригиналната производителност на Thumb. Тези резултати потвърждават, че Thumb-2 отговаря на своите дизайнерски цели.

Фигура 13.13 показва как са кодирани новите 32-битови Thumb инструкции.

Кодирането е съвместимо със съществуващите инструкции за безусловно разклоняване на Thumb, които имат битовата схема 11100 в петте най-леви бита на инструкцията.

Никоя друга 16-битова инструкция не започва с шаблона 111 в трите най-леви бита.

Битовите полета 11101, 11110 и 11111 показват, че това е 32-битова инструкция Thumb.

ARM ПРОЦЕСОР

Тук се разглеждат някои особености на архитектурата и организацията на ARM. ARM е предимно RISC система със следните забележителни атрибути:

- Средно голям масив от еднородни регистри, повече отколкото се намират в някои CISC системи, но по-малко, отколкото в много RISC системи.

- Модел за зареждане/съхранение на обработка на данни, при който операциите се изпълняват само върху операнди в регистри, а не директно в паметта. Всички данни трябва да бъдат заредени в регистри, преди да може да се извърши операция; след това резултатът може да се използва за по-нататъшна обработка или да се съхранява в паметта.

- Единна инструкция с фиксирана дължина от 32 бита за стандартния набор и 16 бита за набора от инструкции Thumb.

- За да направи всяка инструкция за обработка на данни по-гъвкава, или смяна, или ротация може да обработи предварително един от изходните регистри. За ефективно поддържане на тази функция има отделни аритметично логическо устройство (ALU) и модули за превключване.

- Малък брой режими на адресиране с всички адресати за зареждане/съхранение, определени от регистри и полета с инструкции. Не се използват непряко или индексирано адресиране, включващо стойности в паметта.

- Режимите на адресиране с автоматично увеличение и автоматично намаляване се използват за подобряване на работата на програмните цикли.

- Условното изпълнение на инструкции минимизира необходимостта от инструкции за условен преход, като по този начин подобрява ефективността на конвейера.

Организация на процесора

Организацията на ARM процесора варира значително от една реализация до следваща, особено когато се базира на различни версии на ARM архитектурата. Опростена, обща ARM организация на процесора е представена на Фигура 14.25.

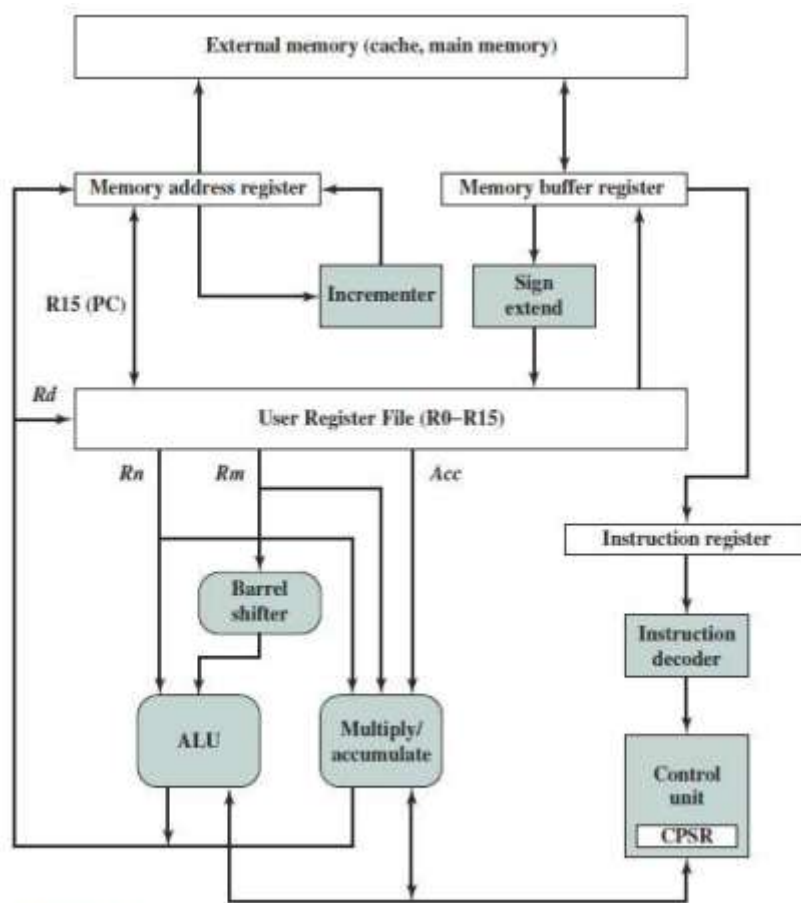


Figure 14.25 Simplified ARM Organization

На тази фигура стрелките показват потока от данни. Всяка кутия представлява функционален хардуерен блок или единица за съхранение. Данните се обменят с процесора от външна памет чрез шина за данни. Прехвърлената стойност е или елемент от данни в резултат на инструкция за зареждане или съхранение, или извличане на инструкция. Извлечените инструкции преминават през декодер на инструкции преди изпълнението си, под управлението на контролен блок. Последният включва конвейерна логика и осигурява контролни сигнали към всички хардуерни елементи на процесора. Елементите с данни се записват в регистров файл, състоящ се от набор от 32-битови регистри. Елементите от байт или полудума, третираны като числа в допълнителен код, се разширяват знаково до 32 бита. Инструкциите за обработка на данни ARM обикновено имат два източника: регистри R_n и R_m и един регистър за резултат/местоназначение, R_d . Стойностите в регистъра на източника се подават в ALU или в отделен умножителен блок, който използва допълнителен регистър за натрупване на частични резултати. ARM процесорът включва също хардуерен блок, който може да измести или завърти стойността на R_m , преди да влезе в ALU. Това изместване или завъртане става по време на цикъла на инструкцията и увеличава мощността и гъвкавостта на операциите по обработка на данни. Резултатите от дадена операция се подават обратно в регистъра на получателя. Инструкциите за зареждане/съхранение могат също да използват изхода на аритметичния блок за генериране на адреса на паметта за зареждане или съхранение.

Режими на работа на процесора

Обикновено процесорът поддържа само малък брой режими. Много операционни системи използват само два режима: потребителски режим и режим на ядрото, като последният се използва за изпълнение на привилегирован системен софтуер. За разлика от тях, ARM архитектурата осигурява гъвкава основа за операционните системи за прилагане на различни политики за защита. Архитектурата ARM поддържа седем режима на изпълнение. Повечето приложни програми се изпълняват в **потребителски режим**. Докато процесорът е в потребителски режим, изпълняваната програма не може да получи достъп до защитени системни ресурси или да промени режима, освен чрез предизвикване на изключение. Останалите шест режима на изпълнение се наричат привилегировани режими. Тези режими се използват за стартиране на системния софтуер. Има две основни предимства за дефинирането на толкова много различни привилегировани режими:

(1) ОС може да приспособи използването на системния софтуер към различни обстоятелства;

(2) определени регистри са предназначени за използване за всеки от привилегированите режими, позволявайки по-бързо промени в контекста.

Режимите на изключение имат пълен достъп до системните ресурси и могат свободно да променят режимите. Пет от тези режими са известни като режими на изключение. Те се въвеждат, когато възникнат специфични изключения. Всеки от тези режими има някои специални регистри, които заместват някои от регистрите на потребителския режим и които се използват за избягване на повреда на информацията за състоянието на потребителския режим, когато възникне изключението. Режимите на изключение са както следва:

■ **Режим супервайзор:** Обикновено в него работи ОС. Въвежда се, когато процесорът срещне инструкция за софтуерно прекъсване. Софтуерните прекъсвания са стандартен начин за извикване на услуги на операционната система на ARM.

■ **Режим на прекратяване/Abort/:** Въвежда се в отговор на грешки в паметта.

■ **Недефиниран режим:** Въвежда се, когато процесорът се опитва да изпълни инструкция, която не се поддържа нито от основното целочислено ядро, нито от един от копроцесорите.

■ **Режим на бързо прекъсване/Fast Interrupt/:** Влиза винаги, когато процесорът получи сигнал за прекъсване от определения източник на бързо прекъсване. Едно бързо прекъсване не може да бъде прекъснато, но бързо прекъсване може да прекъсне нормално прекъсване.

■ **Режим на прекъсване:** Влиза в него когато процесорът получи сигнал за прекъсване от друг източник на прекъсване (различен от бързо прекъсване). Едно прекъсване може да бъде прекъснато само от бързо прекъсване.

Системен режим. Този режим не се стартира от никакви изключения и използва същите регистри, налични в потребителския режим. Системният режим се използва за изпълнение на определени привилегировани задачи на операционната система. Задачите в системния режим могат да бъдат прекъснати от всяка от петте категории изключения.

Регистрова организация

Фигура 14.26 изобразява видимите от потребителя регистри за ARM. ARM процесорът има общо 37 32-битови регистри, класифицирани, както следва:

■ 31 регистъра с общо предназначение. Всъщност някои от тях, като програмните броячи, имат специални цели.

■ 6 регистъра за състоянието на програмата. Регистрите са подредени в частично припокриващи се банки, като текущият режим на процесора определя коя банка е налична. По всяко време са видими 16 номерирани регистри и един или два регистъра на състоянието на програмата, за общо 17 или 18 софтуерно видими регистри.

На фигурата долу:

■ Регистри R0 до R7, регистър R15 (броячът на програмата) и регистърът на текущото състояние на програмата (CPSR) са видими и споделени от всички режими.

■ Регистри R8 до R12 се споделят от всички режими с изключение на бързо прекъсване, което има свои собствени специални регистри R8_fiq до R12_fiq.

■ Всички режими на изключение имат свои собствени версии на регистрите R13 и R14.

■ Всички режими на изключение имат специален регистър на състоянието на запаметената програма (SPSR).

Modes						
Privileged modes						
Exception modes						
User	System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13(SP)	R13(SP)	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14(LR)	R14(LR)	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Shading indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode.

SP = stack pointer CPSR = current program status register

LR = link register SPSR = saved program status register

PC = program counter

Figure 14.26 ARM Register Organization

РЕГИСТРИ ЗА ОБЩО ПРЕДНАЗНАЧЕНИЕ

Регистърът R13 обикновено се използва като указател на стека и е известен също като SP. Тъй като всеки режим на изключение има отделен R13, всеки режим на

изключение може да има свой собствен специален програмен стек. R14 е известен като регистър на връзката (LR) и се използва за задържане на адреси за връщане на подпрограма и връщане в режим на изключение. Регистър R15 е програмен брояч (PC).

РЕГИСТРИ ЗА СТАТУС НА ПРОГРАМАТА

CPSR е достъпен във всички режими на процесора. Всеки режим на изключение също има специален SPSR, който се използва за запазване на стойността на CPSR, когато се появи свързаното изключение. 16-те най-значими бита на CPSR съдържат потребителски флагове, видими в потребителски режим и които могат да се използват, за да повлияят на работата на програмата (Фигура 14.27).

- Флаг Q: използва се за указване дали е възникнало препълване и/или насищане в някои SIMD-ориентирани инструкции.

- J бит: показва използването на специални 8-битови инструкции, известни като инструкции Jazelle, които са извън обхвата на нашата дискусия.

- GE[3:0] битеве: SIMD инструкциите използват битеве [19:16] като флагове по-голямо или равно (GE) за отделни байтове или полудуми от резултата. 16-те най-малко значещи бита на CPSR съдържат флагове за управление на системата, които могат да бъдат променени само когато процесорът е в привилегирован режим. Полетата са както следва:

- Е бит: Контролира зареждането и съхраняването на порядъка на порядъка на данните; игнорирано при извличане на инструкции.

- Битеве за забрана на прекъсване: Битът забранява прекратяването на неточни данни, когато е зададен; битът I деактивира IRQ прекъсванията, когато е зададен; и битът F деактивира прекъсванията на FIQ, когато е зададен.

- Т бит: Показва дали инструкциите трябва да се тълкуват като нормални инструкции ARM или като инструкции Thumb.

- Битеве за режим: Показват режима на процесора.

Обработка на прекъсвания

Както при всеки процесор, ARM включва средства, които позволяват на процесора да прекъсне текущо изпълняващата се програма, за да се справи с възникнали изключения. Изключенията се генерират от вътрешни и външни източници, за да накарат процесора да обработва определено събитие. Състоянието на процесора точно преди обработката на изключението обикновено се запазва, така че оригиналната програма може да бъде възобновена, когато процедурата за обработка на изключение приключи. Едновременно може да възникнат повече от едно изключения. Архитектурата ARM поддържа седем типа изключения. Таблица 14.4 изброява видовете изключения и режима на процесора, който се използва за обработка на всеки тип. Когато възникне изключение, програмата за обработката му се стартира от фиксиран адрес на паметта, съответстващ на типа на изключението. Тези фиксирани адреси се наричат вектори на изключения. Ако има повече от едно прекъсване, те се обработват в приоритетен ред. Таблица 14.4 изброява изключенията, подредени в приоритетен ред, от най-висок към най-нисък. Когато възникне изключение, процесорът спира изпълнението на програмата след текущата инструкция. Състоянието на процесора се запазва в SPSR, който съответства на типа на изключението,

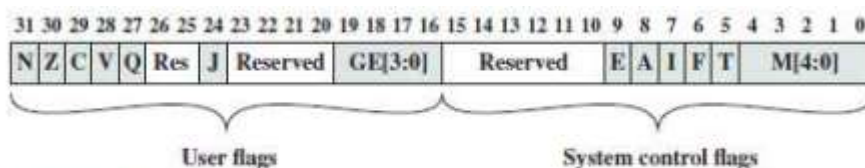


Figure 14.27 Format of ARM CPSR and SPSR

Оригиналната програма може да бъде възобновена, когато процедурата за изключение приключи. Адресът на инструкцията, която процесорът току-що е трябвало да изпълни, се записва в регистъра на връзките за съответния режим на процесора. За да се върне след обработка на изключението, SPSR се премества в CPSR и R14 се премества в компютъра.

Table 14.4 ARM Interrupt Vector

Exception type	Mode	Normal entry address	Description
Reset	Supervisor	0x00000000	Occurs when the system is initialized.
Data abort	Abort	0x00000010	Occurs when an invalid memory address has been accessed, such as if there is no physical memory for an address or the correct access permission is lacking.
FIQ (fast interrupt)	FIQ	0x0000001C	Occurs when an external device asserts the FIQ pin on the processor. An interrupt cannot be interrupted except by an FIQ. FIQ is designed to support a data transfer or channel process, and has sufficient private registers to remove the need for register saving in such applications, therefore minimizing the overhead of context switching. A fast interrupt cannot be interrupted.
IRQ (interrupt)	IRQ	0x00000018	Occurs when an external device asserts the IRQ pin on the processor. An interrupt cannot be interrupted except by an FIQ.
Prefetch abort	Abort	0x0000000C	Occurs when an attempt to fetch an instruction results in a memory fault. The exception is raised when the instruction enters the execute stage of the pipeline.
Undefined instructions	Undefined	0x00000004	Occurs when an instruction not in the instruction set reaches the execute stage of the pipeline.
Software interrupt	Supervisor	0x00000008	Generally used to allow user mode programs to call the OS. The user program executes a SWI instruction with an argument that identifies the function the user wishes to perform.

ARM CORTEX-A8

Последните реализации на ARM архитектурата въведоха суперскаларни техники в конвейера за инструкции. ARM Cortex-A8 е добър пример за RISC-базиран суперскаларен дизайн. Cortex-A8 е в семейството процесори ARM, които ARM наричат процесори за приложения. Процесорът за приложения ARM е вграден процесор, работещ със сложни операционни системи за безжични, потребителски приложения и приложения за изображения. Cortex-A8 е насочен към голямо разнообразие от мобилни и потребителски приложения, включително мобилни телефони, аксесоари, игрови конзоли и автомобилни навигационни/развлекателни системи.

Фигура 16.9 показва архитектурата на Cortex-A8, като посочва потока от инструкции между функционалните блокове. Основният поток от инструкции е през три функционални блока, които реализират двоен, подреден, 13-степенен конвейер.

Cortex-A8. има отделен блок за SIMD (единична инструкция-множество данни) блок, който реализира 10-степенен конвейер.

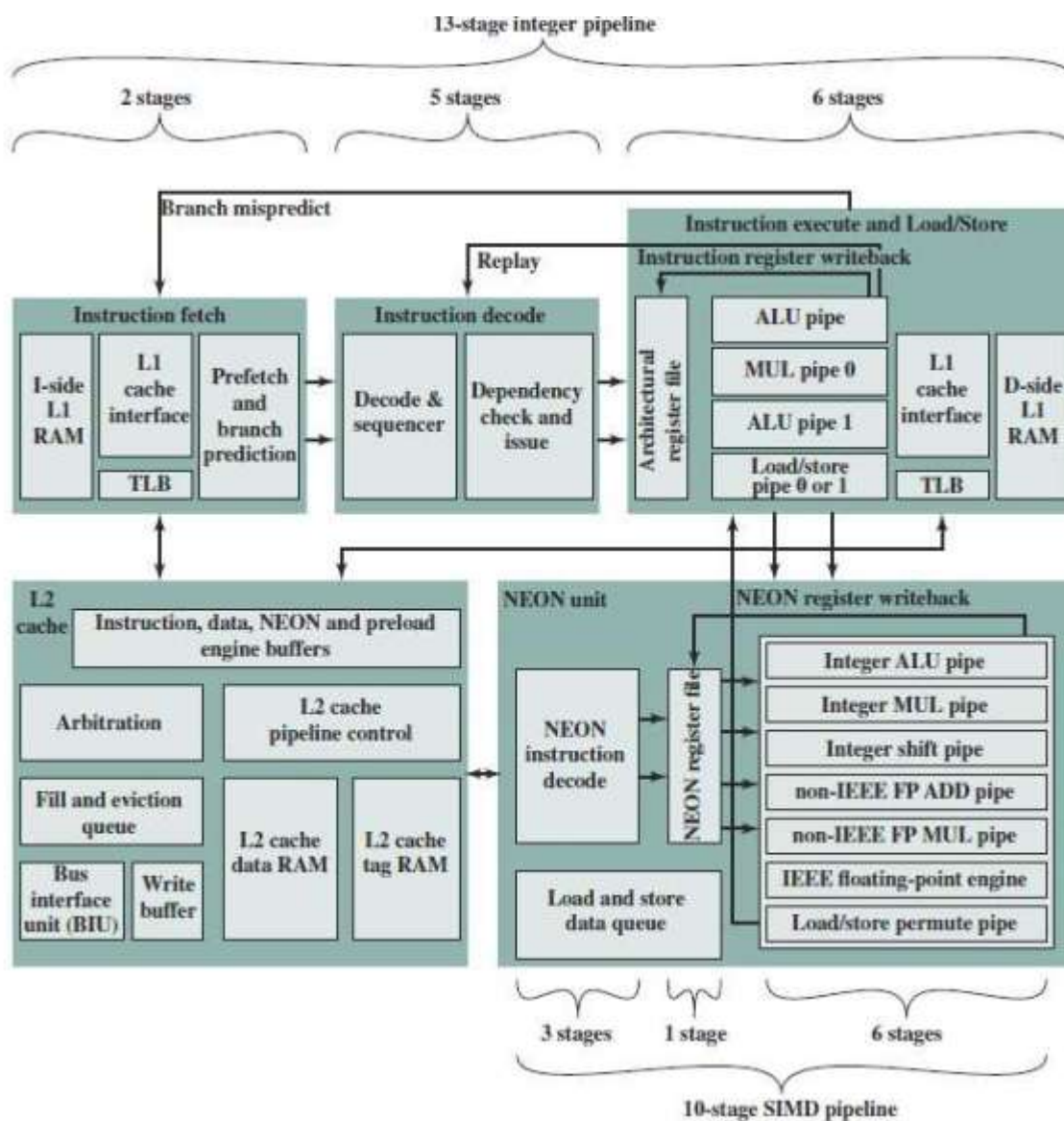


Figure 16.9 Architectural Block Diagram of ARM Cortex-A8

SIMD и Floating-Point конвейър

Всички SIMD и инструкции с плаваща запетая преминават през целочисления конвейер и се обработват в отделен 10-степенен конвейер (фигура 16.11). Това устройство, наречено NEON модул, обработва пакетирани SIMD инструкции и осигурява два типа поддръжка на данни с плаваща запетая. Ако е вграден, векторният копроцесор с плаваща запетая (VFP) изпълнява операции с плаваща запетая в съответствие със стандарта IEEE 754. Ако копроцесорът не присъства, отделните конвейери за умножение и събиране изпълняват операциите с плаваща запетая.

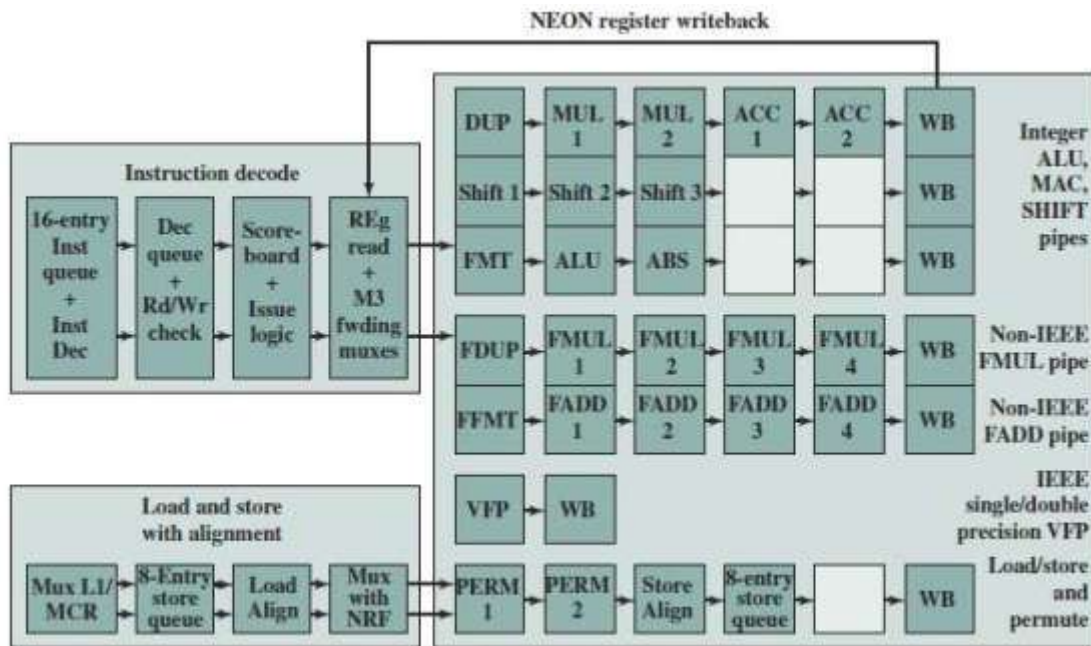


Figure 16.11 ARM Cortex-A8 NEON and Floating-Point Pipeline

ARM CORTEX-M3

Сериата Cortex-M е предназначена за семейството на микроконтролерите. Процесорите Cortex-M трябва да бъдат възможно най-прости и ефективни. Фигура 16.12 предоставя общ преглед на блоковата диаграма на процесора Cortex-M3.

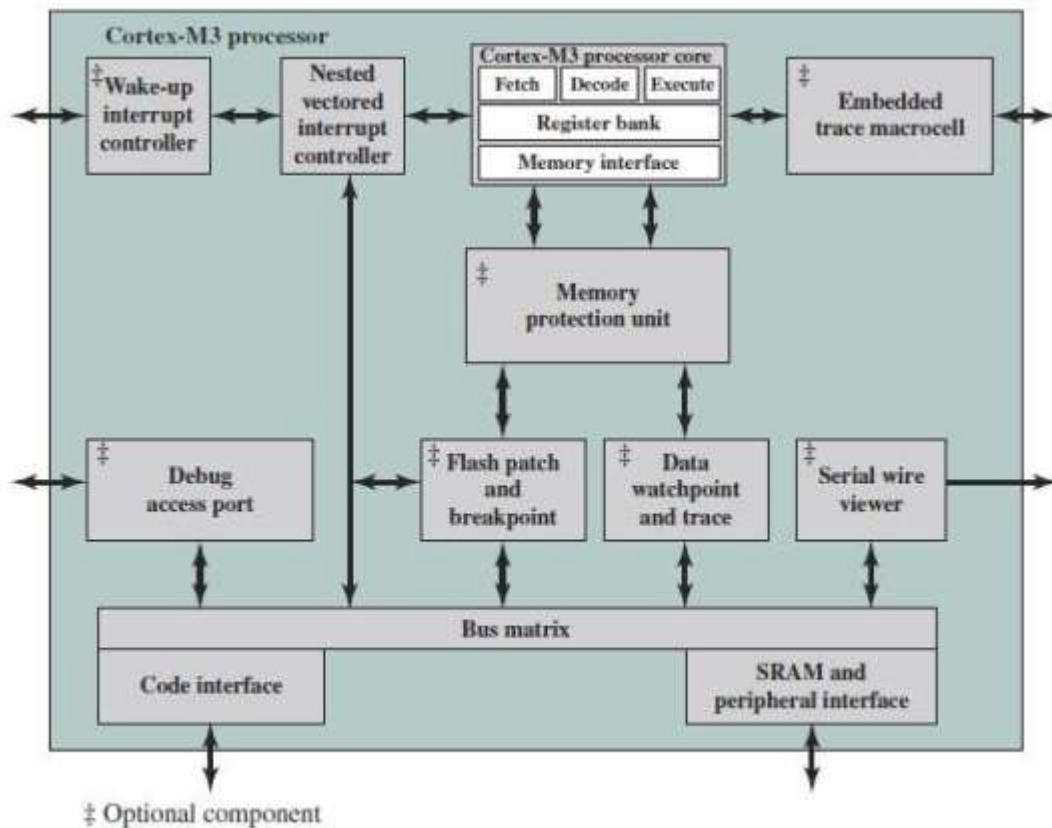


Figure 16.12 ARM Cortex-M3 Block Diagram

- **Ядро на процесора:** Включва тристепенен конвейер, регистърна банка и интерфейс за памет.

- **Устройство за защита на паметта:** Защищава критичните данни, използвани от операционната система, от потребителски приложения, разделяйки задачите за обработка, като забранява достъпа до данните на другия, забранява достъпа до регионите на паметта, позволява на регионите на паметта да бъдат дефинирани като само за четене.

- **Вложен векторен контролер за прекъсвания (NVIC):** Предоставя конфигурируеми възможности за обработка на прекъсвания на процесора. Улеснява обработката на изключения и прекъсвания с ниска латентност и контролира управлението на захранването.

- **Контролер за прекъсване при събуждане (WUIC):** Предоставя конфигурируеми възможности за обработка на прекъсвания на процесора. Той улеснява обработката на изключения и прекъсвания с ниска латентност и контролира управлението на захранването.

- **Модул за флаш корекции и точки на прекъсване:** Слага точки на прекъсване и пачове на код.

- **Точка за наблюдение и проследяване на данни (DWT):** Слага точки за наблюдение, проследяване на данни и профилиране на системата.

- **Сериен преглед на проводници:** Може да експортира поток от генерирани от софтуер съобщения, проследява данни и информация за профилиране чрез един щифт.

- **Порт за достъп за отстраняване на грешки:** Предоставя интерфейс за външен достъп за отстраняване на грешки до процесора.

- **Вградената макроклетка за проследяване** е управляван от приложението източник на проследяване, който поддържа отстраняване на грешки за проследяване на събития в операционната система и приложението и генерира диагностична системна информация.

- **Матричната шина:** свързва интерфейсите за ядрото и отстранява грешки към външните шини на микроконтролера.

ARM CORTEX-A15 MPCORE

Реализирани са хетерогенни многоядрени процесори, използващи ARM ядра, в архитектура big.Little, която използва комбинация от ARM CortexA7 и Cortex-A15 ядра; и архитектура, която комбинира Cortex-A15 ядра с TI DSP ядра.

Многоядрен чип Cortex-A15 MPCore е хомогенен многоядрен процесор, използващ множество от A15 ядра. A15 MPCore е високопроизводителен чип, насочен към приложения, включително мобилни компютри, цифрови домашни сървъри от висок клас и безжична инфраструктура. Ключови елементи на системата са:

- **Общ контролер за прекъсване (GIC):**

Обработка и открива прекъсвания и ги приоритизира. GIC разпределя прекъсванията към отделни ядра.

- **Блок SCU:** Отговаря за поддържане на кохерентността на кеша L1/L2.

GIC е единична функционална единица, която е поставена в системата заедно с A15 ядра. Това позволява броят на поддържаните прекъсвания в системата да бъде независим от дизайна на ядрото A15. Контролните регистри за GIC са дефинирани в адрес на основната памет. Достъп от ядрата на A15 до GIC се осъществява чрез локален интерфейс през SCU.